

Desarrollo con IA Generativa

Tecnología frontend seleccionada

(React + Vite con Tailwind CSS para estilos.)

Los tres modelos de IA evaluados — Gemini, Claude y ChatGPT — analizaron las mismas opciones (Vanilla JS, Vue.js, React, Angular y Next.js) y llegaron a conclusiones convergentes, aunque con matices distintos.

Gemini y Claude recomendaron React con Vite de forma unánime y sin ambigüedad. Ambos argumentaron que la arquitectura basada en componentes es especialmente adecuada para Rentoo, donde la lógica de roles (Admin vs. User) requiere renderizado condicional en múltiples vistas, y donde la gestión del Bearer token de Passport se simplifica con un interceptor de Axios y un contexto de autenticación compartido (**useAuth**). Claude añadió que el dashboard de resumen financiero — con desglose por propiedad, alertas de contratos próximos a vencer y formato monetario — se beneficia directamente del ecosistema de librerías de React (Recharts, Chart.js).

ChatGPT recomendó Vue.js argumentando menor sobrecarga cognitiva para un desarrollador junior, especialmente en formularios complejos como el de contratos con doble inquilino. Sin embargo, el propio ChatGPT reconoció que React es objetivamente superior en demanda laboral, ecosistema y recursos disponibles.

El factor determinante fue la opinión del tutor, quien señaló explícitamente que React es el estándar en los proyectos y empresas del entorno, y que como estudiantes con perfil backend trabajaremos con desarrolladores frontend especializados en React. Esta consideración convirtió el aprendizaje de React en una necesidad práctica más allá del proyecto académico.

En consecuencia, **React con Vite** fue seleccionado como el lenguaje frontend, con **Tailwind CSS** para los estilos, por ser la combinación con mayor relevancia técnica para el proyecto, mayor respaldo del ecosistema, y mayor alineación con el mercado laboral actual.

IA seleccionada

(Claude Sonnet 4.6)

Tras someter el mismo prompt estandarizado a Claude, ChatGPT y Gemini, y complementar ese análisis con comparativas técnicas publicadas y mis propias observaciones, seleccioné **Claude Sonnet 4.6** como la herramienta de IA principal para el desarrollo del frontend de Rentoo.

Proceso de evaluación

Se diseñó un prompt estructurado con ocho deliverables —incluyendo una matriz de recomendación de lenguaje frontend, autoevaluación de la IA, análisis comparativo, plan de arquitectura, roadmap de desarrollo y protocolo de continuidad entre sesiones— y se envió de forma idéntica a los tres modelos.

Los resultados se puntuaron con los siguientes criterios:

Criterion Evaluation	Claude	ChatGPT	Gemini
Usability , Context, Tutoring	15	15	10
Language and AI Gen	34	34	21
Context Continuity Protocol (Highly Important)	10	8	4
TOTAL	59	57	35

Justificación de la elección

Claude y ChatGPT obtuvieron puntuaciones similares en las categorías técnicas. El factor decisivo fue la categoría de Continuidad de Contexto, en la que Claude superó a ChatGPT con una diferencia de 2 puntos. Esta categoría fue ponderada con mayor peso porque el proyecto implica trabajar con una base de código nueva —React + Vite con Tailwind CSS— y aprender el lenguaje al mismo tiempo que se desarrolla. En ese contexto, la capacidad de la IA para:

- Minimizar alucinaciones manteniendo el contexto del proyecto entre sesiones
- Analizar bases de código de múltiples archivos con coherencia a lo largo del tiempo
- Explicar el razonamiento detrás del código, no solo generarlo
- Anticipar casos límite del proyecto sin que se los indique explícitamente

...resulta más crítica que la mera capacidad de generación de código puntual.

Este juicio fue respaldado por el análisis comparativo externo entre GPT-5.3 Codex y Claude Sonnet 4.6 (nxcodes.io, 2026), que concluye que Sonnet 4.6 supera consistentemente a Codex cuando la tarea requiere entender la intención del desarrollador —especialmente con especificaciones vagas o incompletas—, elegir patrones de diseño apropiados sin que se los indique, y producir aplicaciones completas funcionales desde un solo prompt. Los desarrolladores prefirieron Sonnet 4.6 sobre modelos anteriores el 70% de las veces en pruebas reales de Claude Code.

External Sources:

GPT-5.3 Codex vs Claude Sonnet 4.6: La comparación práctica de IA para programación en 2026

<https://www.nxcode.io/en/resources/news/gpt-5-3-codex-vs-claude-sonnet-4-6-coding-comparison-2026>

Claude Opus 4.6 vs. GPT-5.3: Which Writes Better Code?:

<https://www.glbgt.com/hub/claude-opus-4-6-vs-gpt-5-3/>

Finalmente, para un desarrollador junior que está aprendiendo React por primera vez mientras integra una API con autenticación OAuth2, UUIDs, roles de acceso y lógica de negocio específica del mercado español, la calidad pedagógica —explicar el por qué, no solo el cómo— es tan importante como la corrección técnica del código generado. Claude demostró consistentemente esa capacidad a lo largo de la evaluación.

Registro de Interacciones con la IA

Prompt utilizado para la evaluación de la Tecnología frontend y IA

A continuación encontrará el enlace de un prompt trabajado junto a la IA que fue utilizado para evaluar la tecnología front-end, así como para seleccionar la IA. El output de este mismo prompt en Claude sirvió también para crear un proyecto dentro de Claude y rellenar las instrucciones del mismo.



Chat GPT (5.3): <https://chatgpt.com/share/69b16259-9304-8013-9592-8092f9650be7>

Ejemplo de prompts utilizado para ganar esclarecimientos y aplicar correcciones necesarias para ayudar a mantener contexto entre chats.

Durante la configuración y construcción del proyecto de interfaz, surgieron varias situaciones en las que las configuraciones se desviaron de lo esperado o no comprendí del todo las sugerencias. Por lo tanto, era importante aclarar estos puntos para entender el proceso y así poder realizar comprobaciones con recursos distintos a la IA, o para mantener el contexto en conversaciones posteriores. Además estos ejemplos ilustran la capacidad de Claude explicar el razonamiento detrás de las recomendaciones y códigos. Las siguientes imagen ilustra algunos de estos casos.

Ejemplo 1: Aclarando elementos no claros

Please explain the differences between the eslint.config.js generated by Vite 8 and your proposed version, focusing on the defineConfig usage and the relevance of your version to this project.

#eslint.config.js generated by Vite 8

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
```

Show more

Great question. Let me break down the differences clearly.

defineConfig and globalIgnores

The Vite 8 version imports `defineConfig` and `globalIgnores` from `eslint/config` — these are new ESLint 9 helpers that provide TypeScript type hints and a cleaner ignore syntax. They're purely ergonomic wrappers. The underlying config object they produce is identical to a plain array. My version uses a plain array directly, which is equally valid but slightly more manual.

extends array (the important difference)

The Vite 8 version uses:

```
js
extends: [
  js.configs.recommended,
  reactHooks.configs.flat.recommended,
  reactRefresh.configs.vite,
]
```

This is the modern flat config way of composing rule sets — cleaner and more explicit than the legacy `extends: ['eslint:recommended']` string syntax.

My version manually spreads the rules instead:

```
js
...reactHooks.configs.recommended.rules,
```

The Vite 8 approach is actually better here — it's the intended pattern for ESLint 9 flat config.

no-unused-vars rule

The Vite 8 version includes:

```
js
'no-unused-vars': ['error', { varsIgnorePattern: '^[_A-Z_]' }]
```

This treats unused variables as errors (not warnings), but ignores variables starting with uppercase or underscore — useful for React components and constants that are defined but not yet used during development.

My recommendation: keep the Vite 8 version as the base

It's actually superior to what I proposed. You only need to add one line to it — the Prettier integration. Update it like this:



Ejemplo 2: Aclarando sobre lo que se construye en la sesión y explicación para comprensión del código

PROJECT: rentoo-api-frontend
STACK: React + Vite 8, React Router DOM, Axios, Tailwind CSS v3, shadcn/ui (Radix, shadcn@latest v4 CLI, Tailwind v3 compatible)
BACKEND: Laravel 12, Laravel Passport OAuth2, UUID primary keys throughout,
{ "data": ..., "token": "..." } login shape,
{ "data": ... } envelope on all other responses
BASE URL: `http://127.0.0.1:8000/api/v1`
TOKEN STORAGE: `sessionStorage` key: "rentoo_token"
[Show more](#)

Session 2 — API Service Layer Foundation

Welcome back! Let me confirm what we're building today before writing a single line of code.

Milestone: Session 2 — API Service Layer Foundation **Estimated time:** 3-4 hours **Branch to create:** `feature/api-service-layer` cut from `develop`

What we'll build:

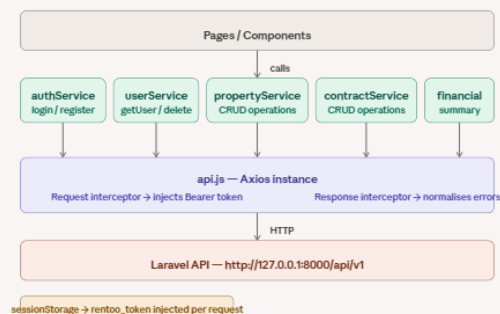
- `src/services/api.js` — Axios instance with interceptors
- `src/services/authService.js` — login, register, logout
- `src/services/userService.js` — getUser, getUsers, deleteUser
- `src/services/propertyService.js` — full CRUD
- `src/services/contractService.js` — full CRUD
- `src/services/financialService.js` — getFinancialSummary
- A temporary verification button in `App.jsx`

Key concepts you should understand by the end: Axios interceptors as middleware, why we centralise the Axios instance, the `{ "data": ..., "token": "..." }` login response shape, and why errors must be re-thrown from the service layer.

Step 1 — `src/services/api.js`

This is the foundation everything else depends on. Build it first, get it right, never touch it from components directly.

Here's how all the pieces connect before we write any code:



Every page talks to a service file. Every service file talks exclusively through `api.js`. `api.js` reads the token from `sessionStorage` and handles all errors once, centrally.

Understanding this code

1. What the Axios instance does and why

Think of `axios.create()` like stamping every envelope before it leaves your house. You configure the return address (`baseURL`), the required paperwork (`Accept: application/json`), and you install two inspectors on the conveyor belt — one that checks outgoing envelopes (the request interceptor) and one that checks incoming packages (the response interceptor). Every service file picks up a pre-stamped envelope; none of them ever think about tokens or base URLs.

2. The request interceptor — why it reads `sessionStorage` fresh every time

Notice the interceptor doesn't store the token in a variable — it calls `sessionStorage.getItem('rentoo_token')` on every request. This is intentional: if the user logs out and a new user logs in in the same browser tab, the old token is gone from `sessionStorage` and the new one is there. If we'd cached the token in a closure variable, the old token would linger.

3. The 401 handler — why `window.location.href` and not `navigate()`

The Axios interceptor lives outside of React — it has no access to React Router's `navigate()` function. Using `window.location.href = '/login'` is the correct pattern here. It also has the side effect of fully refreshing the app, which clears any stale React state from a previous session.

4. Why `authService` returns `response.data` while others return `response.data.data`

Every API response is wrapped in `{ "data": ... }` — except login, which is `{ "data": { ...user..., "token": "..." } }`. The token lives alongside `data`, not inside it. If `authService.login()` returned `response.data.data`, you'd get just `{ id, name, email }` with no token. So auth unwraps one level, everything else unwraps two.

5. The concept to investigate further: Axios interceptors

From `roadmap.md`: Session 2 key concepts: "Axios interceptors as middleware — the request interceptor runs before every outgoing call, the response interceptor runs after every incoming response" A good mental model: `express.use()` middleware in Node — same idea, different direction. Search "Axios interceptors tutorial" to see more patterns.

6. What to test before moving on

- Click the button — does the console show `result.token`? If yes: the Axios instance, the base URL, and the login endpoint all work.
- Open DevTools → Network tab → find the `/login` request → confirm the request headers include `Accept: application/json`
- Temporarily break the base URL (e.g. change port to 9000) → click button → do you see `"Cannot connect to server. Is the API running?"` in the console? If yes: the network error handler works.
- Restore the URL and click again to confirm it recovers

Ejemplo 3: Revisitando el Roadmap para aplicar las correcciones necesarias.

Before we proceed with Session 4, I've been reviewing the roadmap.md and the "UI Design Specifications" instructions and:

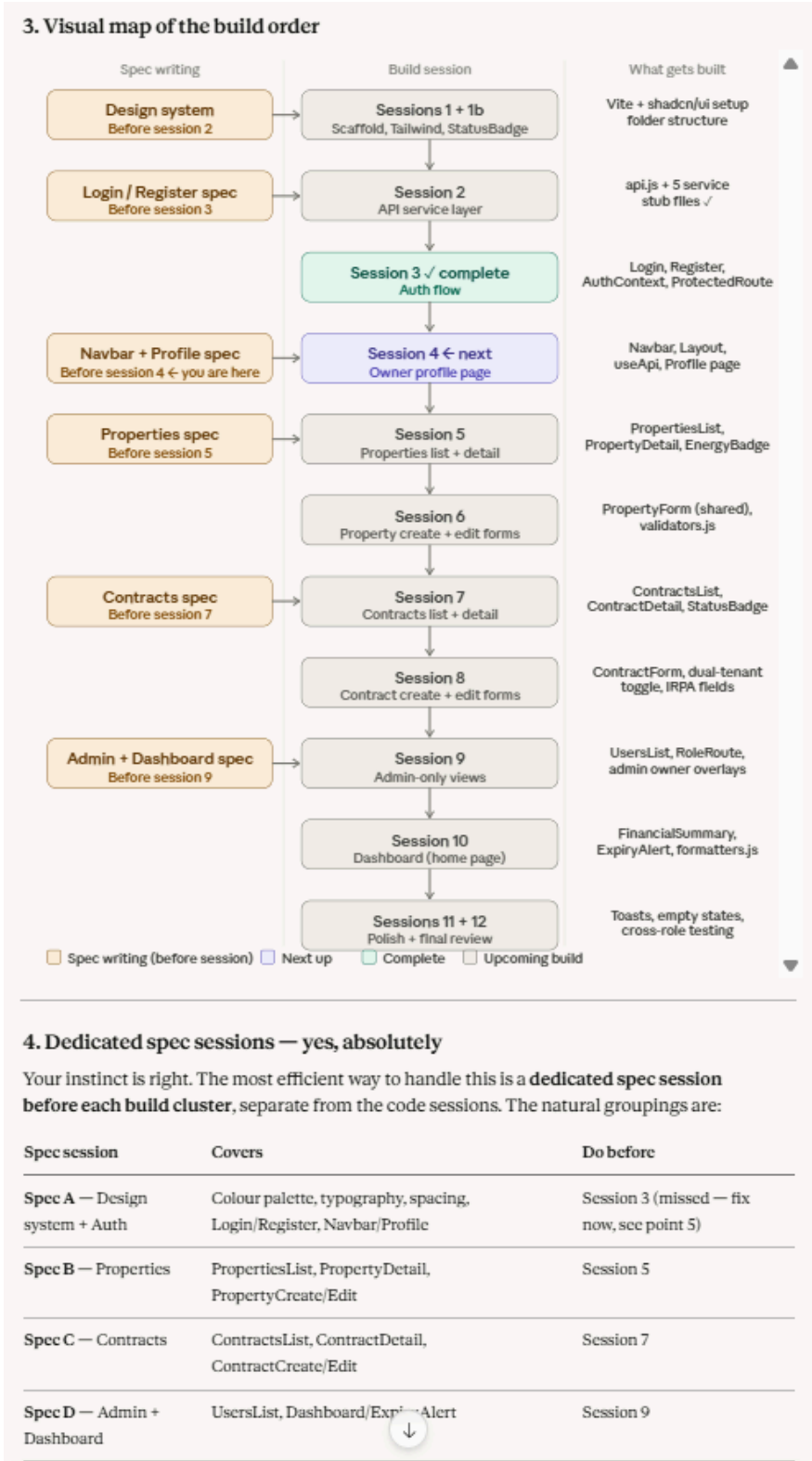
- 1) It's not clear to me where the home page is built. During Session 4 — Owner Profile Page.????
- 2) Could you explain the order of UI construction for pages, components, etc.? Before which session is it recommended to complete the instructions?
- 3) Perhaps in addition to the description in point 2 above, a flowchart or decision tree map visualization would also help to understand the whole scheme more clearly.
- 4) After clarifying the order, I believe it would be very helpful and more efficient to have a dedicated chat before each build session (or a single session for all of them) to co-create the pages and components specifications, ensuring cohesion and that the main project instructions are correctly completed.
- 5) Furthermore, I realized that I did not update the following parts in the User Interface Design Specifications at the correct time. How can we correct this so that everything goes the right way?

```
## UI Design Specifications
> **Instructions:** Fill in each section below before or during
the relevant session. Claude will use these specs to ensure
visual consistency. The Navbar sample shows the expected level of
detail.
### Design System Foundations
*(Fill in before Session 2)*
### Page Specs
#### Navbar / Layout *(sample – use this format for all pages)*
#### Login / Register *(fill in before Session 3)*
#### User / Owner Profile *(fill in before Session 4)*
```

Show less



Me impresionó la capacidad de Claude para generar “flow charts”.
Me facilitó bastante tener esta visualización.



Ejemplo 4: Refinamiento de las especificaciones para “Design System” y “Homepage”

Estos ejemplos, me impresionaron con la capacidad de Claude para generar maquetas. Y segundo mi investigación , esto es super reciente.

Q: The 'Características Principales' section in Miro shows 4 feature pills in a 2x2 grid with coloured icons. Keep this section and content unchanged in React?

A: Option 1, keep all four. However, i'm open to hear about color, typography or icon suggestions that might be more aligned with the aesthetics we are using. This last comment is also valid to the feature cards, since the color purple might cause confusion since is the same color used by Admins. So i'm open for suggestions.

Q: The MVC project had a stat cards row (e.g. '98+ Propietarios', '200+ Propiedades'). These are hardcoded marketing numbers, not live API data. Keep or drop in React?

A: Yes — include stat cards below Características

Q: Navbar on the public welcome page — what should the right side show for unauthenticated visitors? The handoff doc covers the authenticated navbar, but not the public state.

A: Public nav: same full navbar as authenticated but changing the Profile for another generic icon (similar to my profile) where has a Dropdown contents: Sign in (Iniciar sesión), Create an Account (Registrarse), like the CTAs for Estado público (no autenticado). Y can share a screenshot if needed. Additionally, the Dropdown from the user pill should also include a link to the Dashboard (Dropdown contents: header with name + email (read-only), "My profile" link -> /profile, divider, "Dashboard" link ->financial summary, "Log out" in red -> calls authService.logout() then redirects to /login.)

Show less

FEATURE CARDS — PROPOSED COLOUR SYSTEM

Propietarios

Gestiona la información completa de los propietarios: datos personales, DNI/NIE/TIE, datos de contacto y direcciones fiscales.

- ✓ Validación de DNI/NIE/TIE español
- ✓ Gestión de múltiples propiedades
- ✓ Perfiles completos y actualizables

Propiedades

Gestiona el inventario de inmuebles con todos los detalles técnicos, legales y certificaciones exigidas por la normativa española.

- ✓ Referencias catastrales validadas
- ✓ Certificados energéticos y habitabilidad
- ✓ Gestión de gastos (IBI, comunidad)

Contratos

Genera contratos de arrendamiento conformes con la LAU, incluyendo cláusulas específicas y documentos listos para imprimir.

- ✓ Conformes con LAU vigente
- ✓ Cálculo automático de depósitos
- ✓ Documentos imprimibles y firmables

CARACTERÍSTICAS PRINCIPALES SECTION

Características Principales

Cumplimiento Legal

Validación automática de documentos conforme a la normativa española y catalana

Ahorro de Tiempo

Genera contratos completos en minutos, no en horas

Gestión Centralizada

Toda la información de alquileres en un solo lugar

Documentos Profesionales

Contratos listos para imprimir con formato legal estándar

COLOUR SYSTEM — AVOIDING THE PURPLE CONFLICT

blue-600 — Propietarios + User role green-600 — Propiedades indigo-600 — Contratos (proposed)

purple-700 — Admin role only

The indigo-600 for Contratos reads as "legal/document" — close to blue but clearly distinct from both the User blue and the Admin purple. The Características section uses blue, green, indigo, and amber — no purple anywhere on the welcome page.

Q: Contratos entity colour on the welcome page — switch to indigo-600 (#4f46e5)?

A: Yes — indigo-600 for Contratos, drop purple from welcome page entirely

Q: Características Principales icons: blue (Cumplimiento Legal) / green (Ahorro de Tiempo) / indigo (Gestión Centralizada) / amber (Documentos Profesionales) — confirmed?

A: Green and yellow stand out from the more neutral colors. Could we find more muted shades of green and yellow to match the others?

Good eye. The issue is that dcfce7 (green-100) and feF9c3 (yellow-100) are too saturated compared to the softer dbeafe (blue-100) and e0e7ff (indigo-100). Let me find the right muted equivalents.

REFERENCE — ALREADY APPROVED (BLUE + INDIGO)

Cumplimiento Legal

bg #dbeafe / icon #444ed8

Gestión Centralizada

bg #e0e7ff / icon #4f46e5

These two set the saturation target — soft, low-chroma backgrounds.

GREEN OPTIONS — AHORRO DE TIEMPO

A — green-100 (current)

bg #dcfce7 — too vivid

B — emerald-100

bg #d1f7e5 — slightly cooler

C — muted sage

bg #e2f5dc — closest to blue-100 intensity

D — emerald-50

bg #ecfdf5 — very subtle

AMBER / YELLOW OPTIONS — DOCUMENTOS PROFESIONALES

A — yellow-100 (current)

bg #fff9c3 — too bright

B — amber-100

bg #fff176 — warmer tone

C — warm sand

bg #ffccbc — closest to blue-100 intensity

D — amber-50

bg #fff176 — very subtle

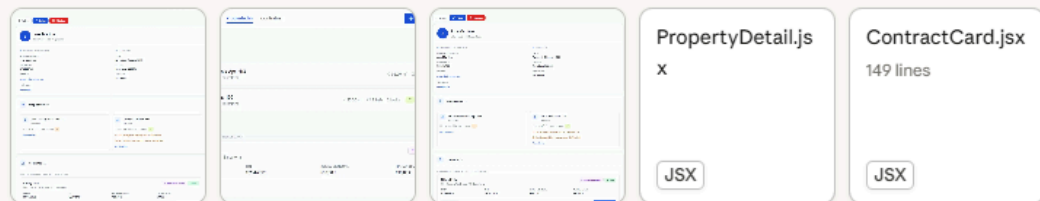
Recommendation: Green C (#e2f5dc) + Amber C (#ffccbc). Both match the low-saturation feel of blue-100 (#dbeafe) and indigo-100 (#e0e7ff) most closely. The four backgrounds together would be: #dbeafe / #e0e7ff / #e2f5dc / #ffccbc — all similarly muted.

My recommendation is Green C + Amber C — they match the visual weight of the blue and indigo backgrounds almost exactly. But pick what reads right to you:

Ejemplo 5: Asegurando consistencia a través de las diferentes vistas.

A pesar de contar con una documentación exhaustiva de roadmap, instrucciones detalladas en el proyecto y sesiones estructuradas entre escritura de especificaciones y construcción, mantener la consistencia visual y funcional entre vistas similares sigue siendo uno de los principales retos del desarrollo asistido por IA, requiriendo una atención constante y revisión crítica por parte del desarrollador. Una vez que la IA generativa tiende a reinventar soluciones ya establecidas si no se le proporciona el contexto exacto en cada interacción.

Contract Card Consistency



Before closing session 5, I want to refactor Section 3 of `PropertyDetail.jsx` to improve the UX. I'd like to maintain consistency with `PropertiesList.jsx` and `Profile.jsx` (variant="full"), while keeping the "Nuevo Contrato para esta propiedad" CTA link. It requires improving the readability and context-specificity of the start date, end date, rent, and fianza legal information.

Note: Simply changing the contracts card to the "full" variant won't work because the added borders would negatively impact the user experience.

Looking at the three screenshots and the attached files, I can see exactly what you mean. The `compact` variant in `PropertyDetail` is a single horizontal row — good for density but the financial info is cramped into a small subtitle line. The `full` variant adds too much visual weight with its card border inside an already-bordered container.

The right solution is a **third variant: "inline"** — no outer card border (so it sits cleanly inside the contracts card), but with the full meta grid layout for readability. This keeps consistency with the data presentation in `PropertiesList/Profile` while fitting the nested context.

Here's the plan:

- Add `variant="inline"` to `ContractCard.jsx`
- Update `PropertyDetail.jsx` to use it
- The "Nuevo Contrato" CTA stays exactly where it is

Forms Consistency:



During testing, I found the following issues:

- The status selector is displaying in English instead of Spanish (see attached image).
- The empty form field alignment doesn't match the blur logic in PropertyForm.jsx and Register.jsx (see attached images and files).
- handleChange and validation are inconsistent with Register.jsx, allowing numbers in name fields and letters in the phone number field. Please align validation logic with Register.jsx.

Good catches. Let me read both files before writing any fixes.

Viewed 2 files >

Good. I have everything I need. Three concrete problems to fix:

1. Status selector shows raw values (`active`, `draft`, `finalized`) instead of the `StatusBadge` — this is because shadcn's `SelectItem` renders its `value` as the trigger text when nothing else is provided. The badge-inside-SelectItem pattern needs a `SelectValue` with a custom render.

2. Input styling is wrong — `ContractForm` uses shadcn's `<Input>` component with Tailwind overrides, but the rest of the project uses plain `<input>` elements with a `getFieldState`/`ic()` pattern that produces three distinct visual states (rest, error, valid). I need to align to that pattern.

3. Name/phone character filtering is missing — `handleChange` needs the same guards Register uses: block digits from `tenant1_name`/`tenant2_name`, block non-numeric from `tenant1_phone`/`tenant2_phone`.

APPENDIX

Evaluation Plan

Criterion Evaluation	Claude	ChatGPT	Gemini
Usability , Context, Tutoring	15	15	10
• Easy of use	5	5	4
• Response/Answers	5	5	3
• Compatibility with the project	5	5	3
Language and AI Gen	34	34	21
• Frontend Language Matrix accurated	5	5	5
• Frontend Language Recommendation Justifications	5	5	4
• AI Matrix Comparison accurated	4	5	3
• Self-Assessment honest	5	5	2
• Architecture plan is actionable	5	5	2
• Layer Description	5	5	2
• Roadmap realistic and detailed	5	4	3
Context Continuity Protocol (Highly Important)	10	8	4
• Prevent Hallucination	5	4	2
• Input requirements accurated	5	4	2

TOTAL

59

57

35